ENGINEERING SPECIFICATION · INTERNAL DOCUMENT

LoadBoardGram — technical documentation

System architecture, data model, AI subsystem, and engineering decisions — from what's already running to what's designed for scale.

Document version **0.1** Date **2026-08-17** Product status **MVP / demo** Related document **Project Overview (for investors)**

01 About this document

This is the technical specification for LoadBoardGram — a freight marketplace that connects shippers and carriers directly, with no broker in between. Unlike the investor overview (`project-documentation.html`), there's no marketing simplification here: the document describes the system the way the engineering team sees it — including what hasn't been built yet.

Audience: technical co-founders, future engineers joining the team, and investor-side technical due diligence.

Every section and every table row carries a maturity label:

`MVP · DONE` — implemented and running in the current demo scaffold· `PHASE 2` — next up in the development sprint· `PHASE 3` — planned, design not yet frozen· `R&D` — needs research/prototyping before it's committed to the product· `RISK` — known technical debt or an open question.

The document deliberately doesn't hide the immaturity of the current codebase: the MVP is an intentionally minimal transactional scaffold (sign up → post a load → take a load → complete it), with zero AI features, built to validate the core shipper↔carrier loop before investing in the ML layer.

02 Overview & maturity

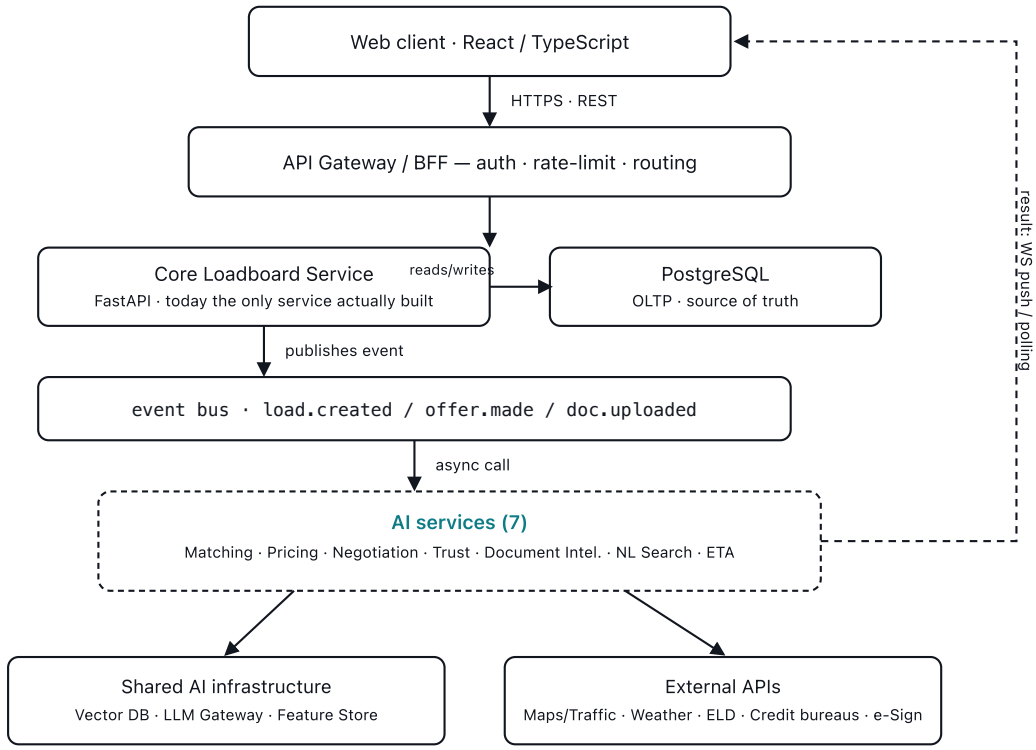
LoadBoardGram is a two-sided platform: a shipper posts a load, a carrier sees it and takes it directly — no broker. The long-term vision isn't just a bulletin board — it's a system where AI does the work of a dispatcher and a broker: matching loads, forecasting rates, running the first round of negotiation, vetting counterparties, and processing documents.

COMPONENT	CURRENT STATE (MVP)	TARGET STATE
Client	React SPA with client-side routing (`react-router`): marketing landing page, load list, load detail page, docs page, manual `fetch`	React SPA + mobile app (React Native), query caching, realtime via WebSocket
Backend	One monolithic FastAPI service	Core service + eight AI services, decoupled via an event bus
Database	One PostgreSQL, 3 tables (`users`, `loads`, `refresh_tokens`), schema under Alembic migrations	PostgreSQL (OLTP) + pgvector/vector store + Redis (cache/queues) + object storage for documents
Authentication	Email + password, short-lived JWT (15 min) + a revocable, rotating refresh token (section 9)	Refresh token in an httpOnly cookie instead of `localStorage`, MFA, SSO for enterprise accounts
Load matching	MVP: deterministic heuristic against the carrier's history (section 7.3)	ML ranking tuned to carrier profile
AI features	3 of 7 — NL Search, Smart Matching, and ETA, all MVP with no ML/embeddings (sections 7.3/7.6/7.7)	7 AI services (section 7)
Infrastructure	Docker Compose, single host	Kubernetes, multiple environments, autoscaling
Observability	None — just `uvicorn` logs to stdout	Metrics, tracing, structured logs, alerting, AI-eval pipelines

> **A deliberate choice, not an oversight.** AI isn't bolted onto a finished bulletin board — the entire architecture below is designed so AI services plug in as separate, independently deployable event consumers, not as calls inside a monolithic controller. This is the project's central architectural bet.

03 Top-level architecture

The system splits into two halves with different reliability and latency requirements. The **Core Loadboard Service** is the transactional core (accounts, loads, statuses, payments down the line): it has to be fast, consistent, and held to strict SLAs. The **AI services** are compute-heavy, depend on external LLM/ML providers, may take seconds to respond, and must never block the transactional path. That's why they're connected asynchronously through an event bus rather than direct synchronous calls.



Legend: solid line — synchronous request; dashed — asynchronous result returned to the client.

Fig. 1 — The transactional core and the AI services are decoupled by an event bus: Core writes to Postgres and publishes an event, AI services consume it asynchronously and return the result to the client on a separate channel (WS/polling), without blocking the main request.

Right now only the rectangle **Core Loadboard Service ↔ PostgreSQL** in the top half of the diagram is actually built — no API Gateway, no event bus, no AI service at all. Everything else is the target architecture the data model (section 5) and event contracts are already being designed around.

Why an event bus instead of direct calls

- **Failure isolation.** If an LLM provider degrades or goes down, posting and taking loads keep working — they're separate failure domains.
- **Independent scaling and deploys.** The Negotiation Agent can ship separately from Core, with its own worker pool and GPU/API budget, without touching transactional code.
- **Swapping models without core downtime.** An LLM provider or scoring algorithm can be changed by pointing a new consumer version at the same event.
- **Audit by default.** Every event is a journal entry — who proposed what and when — which matters for disputes over what the AI "promised" a counterparty.

04 Technology stack

Client

TECHNOLOGY	ROLE	STATUS
React 18 + TypeScript	SPA, component UI	MVP
Vite	Dev server and build	MVP
Manual `fetch` + React Context	API access and auth state (`AuthContext.tsx`)	MVP
react-router-dom	Client-side routing — load list (`/`) and a detail page (`/loads/:id`)	MVP
Leaflet + CARTO Voyager tiles	Map on the load detail page — origin/destination markers and an approximate route	MVP
OpenStreetMap Nominatim	Geocoding origin/destination into coordinates — free, keyless (`frontend/src/geocode.ts`)	MVP
OSRM (public demo server)	Real driving route, distance, and drive time — free, keyless (`frontend/src/routing.ts`); falls back to a straight line if unavailable	MVP
TanStack Query	Request caching, retries, invalidation	PHASE 2
WebSocket client	Realtime load-status updates, AI agent offers	PHASE 2
React Native	Mobile app for carriers (notifications, GPS)	PHASE 3

Backend & data

TECHNOLOGY	ROLE	STATUS
Python 3.12 + FastAPI	REST API, validation via Pydantic	MVP
SQLAlchemy 2.x	ORM, declarative models	MVP
PostgreSQL 16	Primary store (OLTP)	MVP
python-jose + passlib/bcrypt	JWT, password hashing	MVP
Redis	Session cache, rate-limit counters, task queue (Celery/RQ broker)	PHASE 2
pgvector (Postgres extension)	Vector search for NL queries and matching — starting without a separate vector store	PHASE 2
Kafka / Redis Streams	Event bus between Core and the AI services	PHASE 3
S3-compatible object storage	Document storage (BOL/POD/rate confirmation)	PHASE 2
Alembic	DB schema migrations — applied before the app starts (`backend/entrypoint.sh` in Docker, a CI step, a test fixture), not by the app itself	MVP
httpx	Synchronous HTTP calls to Nominatim/OSRM from the backend — ETA MVP (`core/eta.py`, section 7.7)	MVP

AI / ML

TECHNOLOGY	ROLE	STATUS
Claude Haiku 4.5 (structured outputs)	Extracting a structured filter from a free-text query — NL Search (section 7.6)	MVP
LLM Gateway (a LiteLLM-style layer)	Single call point for models across providers, with fallback and a cost meter	R&D
Claude / GPT-class models (function calling)	Negotiation Agent, document structuring	R&D
Sentence-embedding model (e.g. the `bge` / `e5` class)	Vectorizing loads and queries for matching and search	R&D
Gradient boosting (LightGBM/XGBoost)	Market rate forecast, ETA, trust scoring — tabular data, where explainability matters more than deep nets	R&D
OCR + layout model (e.g. the LayoutLM/Textract class)	Reading BOL/POD/rate confirmation	R&D
MLflow / model registry	Model versioning, reproducibility	PHASE 3

Infrastructure & observability

TECHNOLOGY	ROLE	STATUS
Docker + Docker Compose	Local three-container run (db/backend/frontend)	MVP
Kubernetes (managed — EKS/GKE)	Production orchestration, autoscaling AI workers separately from Core	PHASE 3
GitHub Actions	CI: lint, tests, image builds, deploy	PHASE 2
Terraform	Infrastructure as code (network, DB, cluster)	PHASE 3
OpenTelemetry + Prometheus/Grafana	End-to-end latency metrics and tracing, including AI calls	PHASE 3
Sentry	Frontend and backend error tracking	PHASE 2

05 Data domain model

The current schema is two tables, deliberately minimal for the MVP:

```
-- backend/app/models.py — what actually exists today

users(id, email UNIQUE, hashed_password, company_name,
      role ENUM(shipper|carrier), created_at)

loads(id, title, origin, destination, equipment_type DEFAULT 'Dry Van',
      weight_lbs, price_usd NUMERIC(10,2),
      shipper_id FK→users NULLABLE, carrier_id FK→users NULLABLE,
      shipper_name, carrier_name NULLABLE,
      status ENUM(open|accepted|completed), created_at, accepted_at NULLABLE)
```

> **Resolved (ahead of schedule, still within the MVP phase):** `loads` now has `shipper_id` / `carrier_id` as FKs into `users` — a load is tied to the authenticated user who posted or took it, and API-level ownership checks (section 8) rely on exactly that link. `shipper_name` / `carrier_name` stay as a denormalized display cache, now always set server-side from the owner's `company_name` rather than by the client. Was P0 in section 17, now closed.

`accepted_at` (nullable, set server-side by `accept_load`) was added for the ETA MVP (section 7.7) — it's the only notion of "transit started" the system has today, with no telematics or carrier GPS feed to draw one from instead. `users.company_name` is now self-editable by its owner (`PATCH /api/auth/me`, the frontend's personal-cabinet page, section 8) — email and role stay fixed post-registration. Since `loads.shipper_name` / `carrier_name` are a denormalized cache, not a live join, renaming a company doesn't retroactively rewrite the name on loads already posted or accepted under the old one.

Target schema (Phase 2–3)

The extension adds companies (for trust scoring and MC/DOT numbers), offers and negotiation history, documents with extraction results, and market-rate forecast snapshots:

```
CREATE TABLE companies (
  id            BIGSERIAL PRIMARY KEY,
  legal_name    TEXT NOT NULL,
  mc_number     TEXT UNIQUE,      -- Motor Carrier number
  dot_number    TEXT UNIQUE,
  trust_score   NUMERIC(4,1),     -- cached latest score, 0–100
  created_at    TIMESTAMPTZ DEFAULT now()
);

CREATE TABLE offers (
  id            BIGSERIAL PRIMARY KEY,
  load_id       BIGINT REFERENCES loads(id),
  carrier_id    BIGINT REFERENCES users(id),
  price_usd     NUMERIC(10,2) NOT NULL,
  status        TEXT NOT NULL DEFAULT 'pending', -- pending|countered|accepted|rejected|expired
  source        TEXT NOT NULL DEFAULT 'human',   -- human|ai_agent
  created_at    TIMESTAMPTZ DEFAULT now()
);

CREATE TABLE negotiation_messages (
  id              BIGSERIAL PRIMARY KEY,
  offer_id        BIGINT REFERENCES offers(id),
  sender          TEXT NOT NULL,   -- shipper|carrier|ai_agent
  channel         TEXT NOT NULL,   -- chat|email|sms|voice_transcript
  body            TEXT NOT NULL,
  proposed_price  NUMERIC(10,2),
  created_at      TIMESTAMPTZ DEFAULT now()
);

CREATE TABLE trust_scores (
  company_id      BIGINT REFERENCES companies(id),
  score           NUMERIC(4,1) NOT NULL,   -- 0–100
  factors         JSONB NOT NULL,          -- {"payment_history":.., "insurance_valid":.., "doc_fraud_flags":..}
  model_version   TEXT NOT NULL,
  computed_at     TIMESTAMPTZ DEFAULT now(),
  PRIMARY KEY (company_id, computed_at)
);

CREATE TABLE market_rate_snapshots (
  lane            TEXT NOT NULL,   -- e.g. 'DAL–HOU'
  equipment_type  TEXT NOT NULL,
  predicted_at    TIMESTAMPTZ NOT NULL,
  horizon_days    INT NOT NULL,
  predicted_usd   NUMERIC(10,2) NOT NULL,
  confidence_low  NUMERIC(10,2),
  confidence_high NUMERIC(10,2),
  model_version   TEXT NOT NULL
);

CREATE TABLE documents (
  id              BIGSERIAL PRIMARY KEY,
  load_id         BIGINT REFERENCES loads(id),
  doc_type        TEXT NOT NULL,   -- bol|pod|rate_confirmation|insurance_coi
  storage_uri     TEXT NOT NULL,   -- s3://...
  extraction      JSONB,           -- structured fields from the IDP pipeline
  confidence      NUMERIC(4,3),
  review_status   TEXT NOT NULL DEFAULT 'auto', -- auto|needs_review|reviewed
  created_at      TIMESTAMPTZ DEFAULT now()
);
```

`loads.embedding vector(768)` is added as a pgvector column for semantic search and matching (sections 7.3/7.6) — no separate vector database is needed until load volume calls for ANN indexes outside Postgres.

06 Backend services

SERVICE	RESPONSIBILITY	KEY TECH	STATUS
Core Loadboard Service	Users, loads, statuses, offers, payments (future)	FastAPI + Postgres	MVP
Matching Service	Ranking open loads against a carrier's profile and history	MVP: deterministic heuristic (section 7.3); pgvector + gradient-boosted ranking is the target	MVP
Pricing Service	Forecasting the market rate on a lane N days out	LightGBM / time series, snapshots in `market_rate_snapshots`	R&D
Negotiation Service	LLM agent that runs the first round of contact and bargains within set bounds	LLM function calling + guardrails (section 7.1)	R&D
Trust Service	Counterparty trust scoring, double-brokerage flags	Tabular classifier + external checks (FMCSA, insurance)	R&D
Document Intelligence Service	OCR and structuring of BOL/POD/rate confirmation	OCR + layout model + LLM extraction (section 7.2)	R&D
Search Service	Natural-language load search	LLM parsing (`POST /api/search` , section 7.6) → structured filter; vector search not yet built	MVP
ETA Service	Arrival-time estimate	MVP: deterministic heuristic on real distance/duration via Nominatim+OSRM (section 7.7); regression on telematics/traffic/weather is the target	MVP
Notification Service	Push/email/SMS for load status changes, new offers	WebSocket + task queue	PHASE 3

Every AI service is a separate deployment that reads from the shared event bus and writes its result back to Postgres through its own narrow API contract (none has direct access to another service's tables). This is deliberately closer to a modular monolith with clear boundaries than to "microservices for their own sake" — a team of 2–5 engineers can't run 9 independent repos; the boundaries are drawn so they can be physically split apart later without rewriting domain logic.

07 AI/ML subsystem

Seven product-facing AI features on shared infrastructure. The common principle across all of them: **a model is never a single point of failure** — every service has a deterministic fallback (an empty recommendation list, the last known rate, escalation to a human), and no AI call blocks the core transactional path (section 3).

FEATURE	APPROACH	KEY DATA	HUMAN IN THE LOOP
Smart load matching MVP	MVP: deterministic heuristic on equipment type + lanes (embeddings + gradient boosting is the target)	Route, equipment type, the carrier's own history of accepted loads	No — ranking only, the carrier picks
Market rate forecast	Time series / gradient boosting	Historical lane rates, fuel, seasonality, supply/demand	No — confidence interval shown explicitly
AI negotiation assistant	LLM agent with function calling	Price bounds from the shipper, lane history, counterparty messages	Yes — escalation outside trusted bounds (section 7.1)
Trust scoring	Tabular classifier + rules	Payment history, FMCSA data, insurance documents, fraud flags	Yes — a low score blocks auto-matching, needs manual review
Document automation	OCR + LLM extraction	Scanned/photographed BOL, POD, rate confirmation	Yes — below the confidence threshold (section 7.2)
Natural-language search MVP	LLM parsing → structured filter (vector search — planned)	Free-text user query	No — deterministic fallback to a keyword match
ETA forecast MVP	MVP: deterministic heuristic on real distance/duration (regression on telematics/traffic/weather is the target)	Real distance/duration (Nominatim+OSRM), `accepted_at`	No

7.1 AI negotiation assistant

The product's riskiest feature: the model conducts real communication on the platform's behalf and can effectively "promise" a price. That's why it's the only AI service with a hard safety perimeter, not just an LLM call.

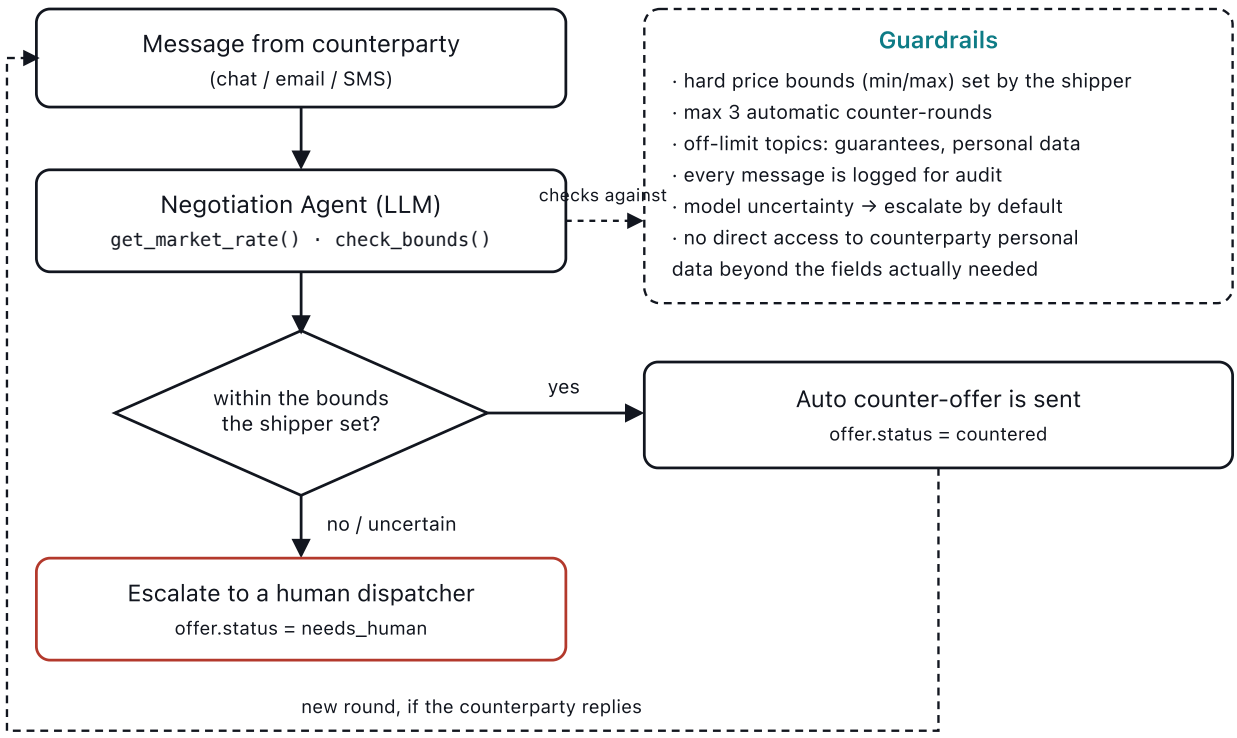


Fig. 2 — The agent only proposes a price within the bounds the shipper set in advance; going outside those bounds, or model uncertainty, always leads to escalating to a human — never to the AI "guessing."

Why it works this way: an LLM, by construction, doesn't guarantee it will respect numeric bounds just because a prompt tells it to — so bound-checking is pulled out into deterministic code (`check_bounds()`) rather than trusted to the model. The agent can shape wording and tone, but it can't get around the hardcoded amount check.

7.2 Document automation

The recognition pipeline for BOL/POD/rate confirmation: `upload → OCR/layout → LLM structuring into JSON → confidence check → auto-file or manual-review queue`. The `confidence` field on the `documents` table (section 5) is the minimum confidence across required fields (amount, date, signature/load number); below the threshold (e.g. 0.90) the document goes into the manual-review queue instead of being applied automatically. Every manual correction is logged as a training example for fine-tuning/prompt tuning — the feedback loop is described in section 13.

7.3 Smart load matching MVP

Shipped: `GET /api/loads/matches` (`api/routes/matching.py`) — a deterministic heuristic, no LLM and no ML model. For a carrier with history (any load where they're the `carrier_id`, any status), every open load gets a score: +3 for an equipment type they've hauled before, +2 for an origin-state match, +2 for a destination-state match against the carrier's own lanes; results sort by score, then by post time. A carrier with no history yet (a fresh account) gets the plain newest-first open list — the same underlying DB query `GET /api/loads?status=open` runs, though `/matches` itself stays a full unpaginated array, unlike that endpoint (section 8) — a deliberate deterministic "cold start" case, not an empty or broken response.

Why not embeddings + gradient boosting yet: the target model below needs real behavioral data (accept/decline history at scale) that doesn't exist in a demo — training on a handful of seed loads would learn noise, not signal. Same principle as NL Search (section 7.6): ship the deterministic version that's actually useful today, upgrade to a model once there's data worth learning from.

7.4–7.7 Other AI services — design principles

`Matching — target model (7.3)`

Load embeddings (route, equipment type, weight, timing) + carrier profile (home base, preferred lanes, accept/decline history) → cosine similarity as the base signal, with gradient boosting on top to calibrate against actual acceptance conversion. Replaces the heuristic above once real data accumulates.

`Pricing (7.4)`

Forecast at the lane level (origin-dest-equipment), 1–7 day horizon, with a confidence interval rather than a point number — the platform shows its uncertainty explicitly instead of passing it off as fact.

`Trust Scoring (7.5)`

0–100 with explained factors (JSONB `factors`), not a black box — the carrier/shipper can see why the score is what it is, and can dispute it. A low score doesn't auto-ban; it lowers matching priority and adds a warning.

`NL Search (7.6)` `MVP`

Shipped: `POST /api/search` (`core/llm.py`) calls Claude Haiku 4.5 via `messages.parse()` with a structured output (`SearchFilter`: origin/destination/equipment_type/price_max/weight_min/weight_max), and the result is applied as ordinary SQLAlchemy filters over `loads`. Vector search by semantic closeness (embeddings) isn't built yet — still a Phase 2/3 target. The LLM call itself is off by default behind a `NL_SEARCH_ENABLED=false` flag in `.env` — so it never spends Anthropic credits until someone deliberately turns it on; `core/llm.py` checks the flag before it even looks at the key. If the flag is off, `ANTHROPIC_API_KEY` is missing, the Anthropic call fails, or the account is out of credit, the fallback is neither a "filter form" nor the unfiltered list — it's a plain keyword match (`_keyword_filter`): every word in the query must `ILIKE`-match at least one of title/origin/destination/equipment_type; the endpoint never errors out on an LLM failure and always filters something.

`ETA (7.7)` `MVP`

Shipped: `GET /api/loads/{id}/eta` (`core/eta.py`) — a deterministic heuristic, no ML model. Geocodes origin/destination via Nominatim, fetches the real driving distance/duration via OSRM (same providers as the frontend's map, called server-side here), and widens OSRM's plain car-profile duration into a `[drive_hours_min, drive_hours_max]` band (×1.3) — a stand-in for typical mandatory rest stops on a long haul. Once a carrier accepts a load (`loads.accepted_at`, set server-side by `accept_load`), the endpoint adds that band to `accepted_at` and returns an arrival window (`eta_earliest` / `eta_latest`); before acceptance it returns the transit-time estimate only, since there's no departure time to anchor a window to. Surfaced on the load detail page as an "Estimated arrival" panel once a load is accepted.
Why not the real model yet: the target model needs carrier GPS telematics, live traffic, and weather — none of which exist in this demo. Same principle as Matching (7.3) and NL Search (7.6): ship the deterministic version that's actually useful today.

`Common to all`

Every model carries a version (`model_version`), logs input/output for offline evaluation, and has a documented fallback to deterministic behavior on failure.

7.8 Shared AI infrastructure

- **LLM Gateway** — a single call point for models: abstracts the provider (swap models without changing service code), meters token cost per request, applies timeouts, and falls back to a smaller/cheaper model if the primary provider fails.
- **PII redaction before outbound calls.** Driver personal data and counterparty contact details are masked before the text goes to a third-party LLM provider, and restored after the response — a requirement for document flows carrying driver names and phone numbers.
- **Prompt registry and eval sets.** Every prompt is versioned; a prompt change is run against a fixed set of real cases (a regression suite for AI, not just for code) before it ships.

- **Canary model rollout.** A new model/prompt version first gets a small slice of traffic, is compared on quality and cost metrics, and only then is scaled to 100%.
- **Cost budget and circuit breaker.** A spend cap on AI calls per period; going over it degrades to cheaper/deterministic paths instead of silently running up the bill. `MVP` NL Search already has the simplest form of this: a daily cap on Anthropic calls (`NL_SEARCH_DAILY_LIMIT`, default 500 per UTC day, `backend/app/core/llm.py`) — once spent, search quietly degrades to the keyword filter. It counts calls, not $/tokens, and lives in process memory (section 17, item 17); a real $ budget and a circuit breaker on provider errors remain the target.
- **Feature Store.** Shared features (lane history, carrier behavior) are computed once and reused by Matching, Pricing, and Trust instead of duplicating the logic.

---

08 API

Conventions

- Base path `/api`, JSON request/response bodies, errors as `{"detail": "..."}` (FastAPI/Pydantic's standard shape).
- Authentication — an `Authorization: Bearer <JWT>` header.
- Rate limiting: `POST /api/auth/register`, `/login`, and `POST /api/search` are limited per IP; over the limit is a `429` with a `Retry-After` header (seconds) and the same `{"detail": "..."}` shape (section 10).
- The public spec is auto-generated by FastAPI: `/docs` (Swagger UI) and `/openapi.json`.
- Planned for Phase 2: path versioning (`/api/v1/...`), idempotency keys for `POST /offers`.
- `GET /api/loads` already paginates — the MVP shipped simple page-number (offset) pagination via `?page=` / `?page_size=` (see below) instead of the cursor pagination (`?cursor=`) this document previously listed as the Phase 2 target. Offset pagination is simpler to implement and gives the familiar "page X of Y" UI, but degrades on very large, frequently-mutated datasets (the offset drifts under concurrent inserts/deletes, and deep-page `OFFSET` queries scan and discard rows). Cursor pagination remains the Phase 2 target for when the load volume actually justifies the more complex implementation.

Endpoints shipped today (MVP)

METHOD	PATH	DESCRIPTION	AUTH
`GET`	`/api/health`	Liveness check	—
`POST`	`/api/auth/register`	Sign up (email, password, company, role) — issues an access+refresh pair	—
`POST`	`/api/auth/login`	Log in — issues an access JWT (15 min) + refresh token pair	—
`GET`	`/api/auth/me`	Current user from the token	Bearer
`PATCH`	`/api/auth/me`	Edit profile — `company_name` only (email/role stay fixed); doesn't retroactively rename `shipper_name` / `carrier_name` on loads already posted/accepted — a denormalized cache, section 5	Bearer
`POST`	`/api/auth/refresh`	Exchange a valid refresh token for a new pair (rotation, section 9)	—
`POST`	`/api/auth/logout`	Revoke a refresh token	—
`GET`	`/api/loads`	List loads, optional `?status=` filter, `?page=` / `?page_size=` pagination (default 1/20, capped at 100) — a page past the last one returns an empty `items` list (200, not 404)	—
`GET`	`/api/loads/mine`	Personal cabinet — every load the caller shows up on, either side (posted as shipper or accepted as carrier), no role gate; unpaginated (a personal list, not the main browsing table)	Bearer
`GET`	`/api/loads/{id}`	Single-load detail — backs the frontend's detail page	—
`POST`	`/api/loads`	Post a load — shipper role only, `shipper_id` comes from the token	Bearer
`POST`	`/api/loads/{id}/accept`	Take a load directly — carrier role only, `carrier_id` comes from the token	Bearer
`POST`	`/api/loads/{id}/complete`	Mark completed — only the load's shipper or the carrier who took it	Bearer
`POST`	`/api/search`	Natural-language load search — LLM parsing into a filter (section 7.6), falls back to a keyword match	—
`GET`	`/api/loads/matches`	Smart load matching — deterministic heuristic against the carrier's history (section 7.3)	Bearer, `carrier` role
`GET`	`/api/loads/{id}/eta`	Arrival time estimate — deterministic heuristic on real distance/duration (section 7.7)	—

Planned endpoints (Phase 2–3)

METHOD	PATH	DESCRIPTION
`GET`	`/api/rates/forecast`	Rate forecast by lane/equipment/date
`POST`	`/api/loads/{id}/offers`	Create an offer (human or AI agent)
`POST`	`/api/offers/{id}/counter`	Counter-offer
`GET`	`/api/companies/{id}/trust-score`	Trust score with explained factors
`POST`	`/api/loads/{id}/documents`	Upload BOL/POD/rate confirmation
`WS`	`/ws/notifications`	Realtime notifications: new offers, status changes

09 Authentication, authorization & roles

Today: email+password → `bcrypt` hash → a short-lived access JWT (HS256, `sub` =email, 15 min by default) via `python-jose` plus a separate refresh token (an opaque random string, stored in the DB only as a SHA-256 hash, `refresh_tokens` table). It rotates on every use (`POST /api/auth/refresh`) — the old refresh token dies the instant a new one is issued, so a stolen token that gets replayed after the legitimate client has already refreshed is a rejected reuse, not a silent second session. `POST /api/auth/logout` revokes it outright. The access JWT itself still can't be revoked (stateless by design), but the 15-minute window bounds the blast radius far more than 7 days did. The frontend refreshes the access token in the background, invisibly to the user (`AuthContext.tsx`). Both tokens live in `localStorage` — a deliberate simplification, not the target design (see below).

ROLE	CAN DO TODAY	TARGET (PHASE 2+)
shipper	Post a load	+ see offers, negotiate, see the carrier's trust score
carrier	Take an open load	+ AI recommendations, counter-offer, upload POD
dispatcher (enterprise)	—	Manages multiple trucks/drivers on behalf of a carrier company
admin	—	Moderation, audit-log access, manual document/score review

Target improvements: the refresh token moves to an httpOnly cookie instead of `localStorage` (closes the XSS-read exposure, needs CSRF protection alongside it), revocation moves to a Redis deny-list instead of a Postgres table (scale), MFA for enterprise accounts, SSO (OAuth2/OIDC) for large shipper accounts, scoped API keys for integrations (TMS connectors).

10 Security

AREA	CURRENT STATE	TARGET
Secrets	`JWT_SECRET` from an env var, code default `"dev-secret-change-me"`	A secrets manager (Vault/AWS Secrets Manager), no defaults in code
CORS	An explicit allowlist via `CORS_ORIGINS`	Production frontend domains replace the default dev origin
Rate limiting	In-memory sliding window per client IP (`backend/app/core/rate_limit.py`): `/api/auth/register` + `/login` share a 10/min budget (`AUTH_RATE_LIMIT_PER_MINUTE`), `POST /api/search` gets 20/min (`SEARCH_RATE_LIMIT_PER_MINUTE`); over the limit is HTTP 429 with `Retry-After`. `/refresh` is deliberately exempt (a 256-bit token, nothing to guess). State is per-process, not shared (section 17, item 17)	At the API Gateway or a shared store (Redis), tighter specifically on `/auth/*`
Resource-level authorization	In place on the three mutating load endpoints (section 17, closed)	Same coverage extended as new resources appear (offers, documents)
PII & documents	N/A — no documents yet	Encryption at rest (S3 SSE), PII masking before LLM calls (section 7.8)
Audit	None	`audit_log`: who/what/when, plus a separate log of every AI message sent on the platform's behalf (section 7.1)
Dependencies	Not scanned	Dependabot/ `pip-audit` in CI, blocking on critical CVEs
Pentest	Not done (demo stage)	Before public launch, then yearly / after major changes

11 Infrastructure & DevOps

Today: `docker-compose.yml` brings up three containers — `db` (Postgres 16 with a healthcheck), `backend` (FastAPI, retries the DB connection on startup), `frontend` (Vite dev server). One host, no separate environments.

ENVIRONMENT	PURPOSE	STATUS
local	Development via `docker compose up`	MVP
staging	Pre-release verification, synthetic data	PHASE 2
production	Kubernetes, separate node pools for Core and AI workers (AI workers get GPU access/higher CPU limits and their own autoscaling on event-queue depth)	PHASE 3

CI/CD (Phase 2): GitHub Actions — lint (ruff/eslint) → tests → build Docker images → push to registry → auto-deploy to staging, manual approval for production. **Backups:** a daily Postgres snapshot plus point-in-time recovery up to 7 days; documents in object storage — versioning is on to guard against accidental overwrites.

12 Performance & scalability

- **Caching.** Redis for the "hot" list of open loads and rate-forecast results (a few minutes' TTL — a forecast shouldn't be recomputed on every list render).
- **Async work.** Heavy AI calls (negotiation, OCR) run through a queue (Celery/RQ on Redis, later Kafka), not inside the API's request-response cycle.
- **DB indexes.** A composite `loads(status, created_at)` index for the main list; `loads(origin, destination, equipment_type)` for matching; a pgvector HNSW index on `embedding`.
- **Read replicas.** Load-list and analytics reads come from a replica; writes stay on primary — once load justifies it.
- **CDN.** Frontend static assets and uploaded documents (via signed URLs) sit behind a CDN, not proxied through the backend.

METRIC	TARGET BUDGET
Load list, p95	`< 200 ms`
Post a load, p95	`< 300 ms`
AI matching (async), p95	`< 3 s`
Negotiation Agent first move, p95	`< 8 s`
Core API availability	`99.9%`

13 Observability

Today — just `uvicorn`'s stdout logs, nothing structured. The target stack:

- **Logs** — structured (JSON), carrying a `request_id` through Gateway → Core → event → AI service, for end-to-end tracing of a single user request.
- **Metrics** — Prometheus: per-endpoint latency, event-queue depth, AI call cost in dollars per day.
- **Tracing** — OpenTelemetry, with AI spans tagged separately (model, token count, provider latency).
- **AI-specific observability:** an eval pipeline against real cases after every prompt/model change; quality drift (a sudden jump in the Negotiation Agent's human-escalation rate signals a regression); a human review queue for documents and scoring as an explicit UI, not a side effect.
- **Feedback loop.** Every manual correction (a dispatcher fixing an AI offer, a human correcting a document field) is logged as a training example for the next eval set and potential fine-tuning/prompt tuning.

14 External integrations

CATEGORY	WHY	USED IN
Mapping / traffic	Distance, drive time, congestion	ETA, Matching (deadhead miles)
Weather	ETA adjustment, delay risk	ETA
Carrier ELD / telematics	Real GPS position, driver HOS limits	ETA, Trust
FMCSA / credit bureaus	MC/DOT verification, insurance coverage, claims history	Trust Scoring, company onboarding
e-Signature	Signing rate confirmations without emailing PDFs back and forth	Document Intelligence, offers
Payments / factoring	Fast carrier payout after POD (future monetization)	Load completion
SMS / email provider	Notifications, a channel for the Negotiation Agent	Notification, Negotiation
DAT/Truckstop (public APIs where available)	Cold-start market data for Pricing before enough proprietary history accumulates	Pricing Service

15 Testing & quality

LEVEL	TOOL	COVERS	STATUS
Unit (backend)	pytest	Schemas, business rules (load status transitions), geocoding/routing (mocked `httpx` calls) — 99% line coverage of `app/`, 69 tests	MVP
Unit (frontend)	Vitest + Testing Library	First slice — `AuthContext` (bootstrap via refresh, logout), `EtaWindow`, `AuthPanel`, `haversineMiles`, `Combobox` (search/debounce/keyboard), `LoadsPage` (the post-a-load form's state→city search), `api.ts` (every backend call, including error handling), `App.tsx` (header/nav, auth status, the login modal, routing), `HomePage` (load stats, auth-gated CTAs), `LoadDetailPage` (loading/error states, detail fields, the accept flow's every branch), `DocsPage` (both doc cards, each card's HTML/PDF links, the safe `target="_blank"` / `rel="noopener noreferrer"` attributes), `RouteMap` (Leaflet map — `react-leaflet` mocked at the module level so jsdom never has to render a real map; the loading/unavailable/ready state machine, distance/duration formatting, solid-vs-dashed real-route-vs-straight-line-fallback branching, and the stale-response guard on prop changes, 100% line/branch coverage), and `ProfilePage` (the personal cabinet — profile fields, editing the company name, the caller's own loads list with per-status counts): 13 files, 106 tests — every page/component this tech-debt item originally named by name now has at least some coverage	MVP
Integration	pytest + a real Postgres in Docker	API endpoints end-to-end through the DB — the same test suite as Unit above; the project doesn't split these into separate suites yet	MVP
E2E	Playwright	Sign up → post → take a load	PHASE 3
AI eval	An in-house harness over a fixed case set	Negotiation/Matching/Trust quality — regression checks before every prompt or model release	R&D
Load	k6/Locust	Latency under peak load, event-queue behavior under backpressure	PHASE 3

Today: `backend/tests` — 11 files, 69 tests, 99% line coverage across `app/` (every `api/routes/*` module and `core/eta.py` at 100%): the health check; auth/role/ownership on the three mutating load endpoints plus the full status transition (`open→accepted→completed`, including that a load can't be completed without being accepted first — that check was missing until this pass, closed in the same change, section 17); `POST /api/search` (the Anthropic call mocked at the import site, including its own failure branch — tests are deterministic, no network and no `ANTHROPIC_API_KEY` needed); `GET /api/loads/matches` (ranking, role gating, cold start); `GET /api/loads/{id}` (detail, 404, a regression test that `{load_id:int}` doesn't swallow `/matches`); `GET /api/loads/{id}/eta` at two layers — the route itself (`estimate_transit` mocked) and the heuristic underneath it (`core/eta.py`, `httpx.get` mocked — the actual geocoding/routing/caching/fail-closed logic runs for real); the full refresh-token lifecycle (issuance, rotation, replay rejection, revocation); `PATCH /api/auth/me` (company-name change, 422 on a blank value, and a regression test that renaming doesn't retroactively rewrite `shipper_name` on loads already posted); and `GET /api/loads/mine` (requires a token, a shipper sees only their own posted loads, a carrier only their own accepted ones, newest-first ordering). Frontend: Vitest + Testing Library, a first slice of 13 files/106 tests (see table above); every page/component named as a gap in section 17 now has at least some coverage, though coverage within individual files is still partial.

16 Technical roadmap

PHASE	TECHNICAL GOAL	AI FEATURES UNLOCKED
MVP DONE	Transactional scaffold: auth, loads, statuses	Natural-language search (basic version — structured filter, no vector search yet) and smart load matching (basic version — deterministic heuristic, no ML) DONE
Phase 2 3–4 MO	Resource-level authorization, tests, CI, task queue, object storage, pgvector	Embeddings + gradient boosting for Matching, vector search on top of NL Search, rate limiting/cost cap for LLM calls
Phase 3 5–9 MO	Kubernetes, event bus, observability, mobile app	Rate forecast, trust scoring, document automation, ETA
Phase 4 R&D	Multi-region, enterprise SSO, payment rail	AI negotiation assistant (the highest-value but riskiest feature — shipped last, after users trust the simpler AI features)

The order is deliberate: the Negotiation Agent is the most valuable feature, but also the riskiest — the model speaks on the platform's behalf with real money on the line. It ships last, once the guardrail, audit, and escalation infrastructure (sections 7.1, 13) has already been proven on lower-risk AI features.

17 Technical debt & known limitations

#	ISSUE	IMPACT	PRIORITY
1	~~`POST /api/loads`~~, ~~`/accept`~~, ~~`/complete`~~ ~~didn't check authentication or role~~ — closed: all three now require a Bearer token and check role (section 8)	Was: anyone without a token could post/take/close loads	CLOSED
2	~~`loads.shipper_name`~~ / ~~`carrier_name`~~ ~~are free text, not FKs into~~ ~~`users`~~ — closed: added `shipper_id` / `carrier_id` (section 5)	Was: load ownership couldn't be verified; the data was trivial to fake	CLOSED
3	~~JWT has no refresh and no revocation~~ — closed: a short-lived (15 min) access JWT plus a separate, revocable refresh token with rotation (section 9, `backend/app/core/security.py`). The default `JWT_SECRET` in code remains — a deliberate demo-only exception (security.md), don't forget to replace it before any shared/staging environment (see README)	Was: a stolen token stayed valid for 7 days with no way to revoke it	CLOSED
4	~~CORS~~ ~~`allow_origins=["*"]`~~ — closed: an explicit allowlist via the `CORS_ORIGINS` env var (`backend/app/core/config.py`, `.env.example`) was already in place by the time this table last got a look — the row just wasn't updated then (section 10)	Was: any origin could make cross-origin requests against the API	CLOSED
5	~~The DB schema is created via~~ ~~`Base.metadata.create_all`~~ ~~, no migrations~~ — closed: Alembic (`backend/migrations/`), applied before the app starts in every environment (section 4)	Was: no managed path for evolving the schema against production data	CLOSED
6	~~Backend tests are thin, frontend has no test runner at all~~ — closed for the backend: 99% line coverage of `app/`, 69 tests (found and closed a real bug along the way: `complete_load` let a load be completed without ever being accepted). Closed the infrastructure, not the coverage, for the frontend: Vitest + Testing Library wired into CI, a first slice of 13 files/106 tests (`AuthContext`, `EtaWindow`, `AuthPanel`, `haversineMiles`, `Combobox`, `LoadsPage`, `api.ts`, `App.tsx`, `HomePage`, `LoadDetailPage`, `DocsPage`, `RouteMap`, `ProfilePage`), section 15 — every page/component named here originally now has at least some coverage	Was: most business logic had no regression safety net; coverage within individual files is still partial, and any new page starts from zero again	P1
7	~~No rate limiting on~~ ~~`/api/auth/*`~~ — closed: `/register` and `/login` share one per-IP budget (10/min by default, `AUTH_RATE_LIMIT_PER_MINUTE`), over it is a 429 with `Retry-After` (section 10, `backend/app/core/rate_limit.py`). `/refresh` is deliberately outside the budget: a 256-bit token can't be guessed, and a 429 there would read to the frontend as a dead session and log the user out. The implementation's own limits (in-process state) are tracked separately as item 17	Was: vulnerable to password brute-forcing	CLOSED
8	4 of 7 AI services are still a spec, not code (NL Search, Matching, and ETA MVP are built, sections 7.3/7.6/7.7 — all three deterministic, no ML/embeddings)	Most of the core value proposition for investors hasn't been built yet	OPEN
9	~~`POST /api/search`~~ ~~has no rate limiting and no per-request/per-period cost cap on Anthropic calls~~ — closed: a per-IP 20/min limit (`SEARCH_RATE_LIMIT_PER_MINUTE`) plus a global daily cap on Anthropic calls (`NL_SEARCH_DAILY_LIMIT`, 500 per UTC day), past which search degrades to the keyword filter rather than erroring (section 7.8). `max_tokens` already bounded each request. Section 7.8's full $ budget and circuit breaker remain the target; the "in process memory" limitation is item 17	Was: anyone, unauthenticated, could trigger unlimited billed LLM calls	CLOSED
10	`frontend/src/geocode.ts` calls Nominatim directly from the browser, no backend proxy, no rate limiting beyond an in-memory per-session cache — Nominatim's usage policy caps free use at roughly 1 request/second	Real production traffic risks getting the IP blocked by Nominatim	P2

#	ISSUE	IMPACT	PRIORITY
11	`frontend/src/routing.ts` calls OSRM's public demo server directly from the browser — same shape of gap as item 10: no backend proxy, no rate limiting beyond an in-memory session cache; the server isn't meant for production traffic	No SLA/availability guarantee for real traffic	P2
12	`backend/app/core/eta.py` also calls Nominatim and OSRM's public demo server directly (from the backend now, for the ETA MVP, section 7.7) — same shape of gap as items 10–11, but now server-side traffic too; a process-lifetime in-memory cache (not per-session) softens the load somewhat but is no substitute for real rate limiting or a paid provider	Same blocking/no-SLA risk as items 10–11, now from the server side as well as the client	P2
13	The refresh token (section 9) lives in `localStorage`, same as the access token before it — readable by any JS running on the page. Rotation (item 3 above) limits a stolen token to one use, but doesn't prevent the theft itself; the target design is an httpOnly cookie, invisible to client-side JS entirely, plus CSRF protection to go with it	An XSS anywhere on the site can steal a session for the refresh token's full lifetime (30 days by default), not just the access token's 15 minutes	P2
14	~~`frontend`'s `vite@^5.4.0` transitively pulled a vulnerable `esbuild` (GHSA-67mh-4wv8-2f99)~~ — closed: `vite` bumped `^5.4.0` → `^8.2.1` (with `@vitejs/plugin-react` → `^6.0.5` and `vitest` / `@vitest/coverage-v8` → `^4.1.10` — all four had to move together, section 15). `npm audit` now reports 0 vulnerabilities; Vite 8 doesn't depend on `esbuild` at all anymore, not just a patched version of it. Verified: full test suite, lint, build, and the Docker image (`node:20-alpine` → Node 20.20.2, above Vite 8's `^20.19.0` floor) all pass with no other code changes needed	Was: the dev server accepted cross-origin requests and echoed the response back	CLOSED
15	`frontend/src/placeSearch.ts` calls Photon (`photon.komoot.io`, Komoot's keyless public geocoder) directly from the browser — live, debounced city/town/village search for the post-a-load form's city picker. Same shape of gap as the geocode.ts/routing.ts/core-eta.py items above: no key, no backend proxy, no rate limiting beyond an in-memory per-session cache, and it's explicitly a public demo instance with no uptime or rate guarantee. Deliberately a different provider than Nominatim (used elsewhere for exact geocoding): Nominatim doesn't do true prefix/typeahead matching (verified directly — querying "Chicag" while still typing "Chicago" returns nothing), Photon is built for exactly that autocomplete use case	A fourth third-party mapping service the app now depends on with no production-grade guarantee behind any of them (Nominatim ×2 call sites, OSRM ×2, now Photon)	P2
16	`GET /api/loads` gained pagination (`?page=` / `?page_size=`, section 8) — `GET /api/loads/matches` (matching.py) and `POST /api/search` (search.py) were deliberately left out of that change and still return a full, unpaginated array: both query the DB directly, not through the list endpoint. Fine at demo scale, but as load volume grows both responses grow unbounded — the same class of problem pagination just closed for `GET /api/loads`	Response size and query time for these two endpoints grow without bound as the load count grows	P2
17	Rate limiting (items 7, 9) and the NL Search daily cap live in process memory: under N uvicorn workers/replicas the effective limit is N × the configured one, and a restart resets the counters. The key is the client's TCP address; `X-Forwarded-For` is deliberately ignored (it's client-controlled), so behind a reverse proxy or Docker Desktop's port-forwarding every client may share one address and one budget. The target is an API gateway or Redis behind the same interface, plus trusted-proxy config	Limits are weaker than configured once horizontally scaled; behind a proxy everyone shares one budget	P2

Items 1–2 are closed, ahead of AI work as planned: without an FK link between a load and its owner, trust scoring, personalized matching, and a negotiation log couldn't have been built correctly — the entire AI subsystem in section 7 relies on exactly that link.